SIMULADOR DE ARQUITETURA DE COMPUTADORES

1. VISÃO GERAL DO PROJETO

O que você vamos construir nesse projeto

- CPU Simulator com Fetch, Decode, Execute, ULA, UC, Registradores e FLAGS
- Memory Monitor com Text, Data, BSS, Heap e Stack segmentos
- Integração real de C chamando funções Assembly (inline e externo)
- Interface gráfica Web moderna (HTML + CSS + JavaScript ou React)
- Gráficos em tempo real de uso de CPU, Heap, instruções por segundo
- Simulação didática de ataques (Buffer Overflow, Stack Smashing)
- Documentação acadêmica completa

1.1 Mapa das 10 Fases de Desenvolvimento

- FASE 1 – Planejamento e Requisitos
- FASE 2 – Ambiente de Desenvolvimento
- FASE 3 – CPU Simulator (núcleo lógico)
- FASE 4 – Memory Monitor (núcleo lógico)
- FASE 5 – Integração C + Assembly
- FASE 6 – Interface Visual Web
- FASE 7 – Gráficos em Tempo Real
- FASE 8 – Simulação de Ataques
- FASE 9 – Testes e Validação
- FASE 10 – Documentação Acadêmica e Entrega Final

2. TECNOLOGIAS, FERRAMENTAS E SETUP

2.1 Stack Tecnológica – Escolhas e Justificativas

Tecnologia	Por que usar e quando entra no projeto
C (linguagem)	Linguagem base para lógica do simulador. Controle total sobre memória, ponteiros e operações de baixo nível. Indispensável para implementar malloc, stack frames e ULA.

Assembly x86-64	Integração real com registradores do processador. Mostra ao aluno como operações de baixo nível realmente funcionam. Chamado via inline asm ou arquivos .asm externos.
Python	Backend web (FastAPI ou Flask) para expor a API REST que o frontend consulta. Excelente para prototipação rápida e marshaling de dados do C para JSON
JavaScript/HTML	Frontend da interface visual. Sem framework obrigatório — vanilla JS é suficiente para iniciar. React pode ser introduzido na Fase 6 para componentização.
Chart.js / D3.js	Bibliotecas para gráficos em tempo real (uso de memória, CPU usage, instruções/s). Chart.js é mais simples; D3.js oferece mais controle para visualizações personalizadas.
WebSocket	Protocolo de comunicação em tempo real entre backend Python e frontend. Essencial para atualizar gráficos sem polling constante.
GCC / GAS	Compilador C (gcc) e Assembler GNU (as). Compilam o núcleo do simulador. Geram binários e bibliotecas compartilhadas (.so) consumidas pelo Python via ctypes.
NASM	Netwide Assembler: sintaxe Intel mais legível para os exemplos Assembly x86-64. Complementa o GAS.
ctypes (Python)	FFI do Python para carregar e chamar funções compiladas em C diretamente, sem reescrever lógica.
pytest / CUnit	Testes unitários. pytest para o backend Python; CUnit para as funções C do simulador.

2.2 Ferramentas para Instalar

Instalar na ordem abaixo (Linux/ WSL / macOS)

1. sudo apt update && sudo apt install gcc nasm gdb valgrind build-essential
2. sudo apt install python3 python3-pip python3-venv
3. pip install fastapi uvicorn websockets pytest

4. Node.js 18+ (nodejs.org) – para ferramentas frontend (Vite, Chart.js, etc.)
5. npm install -g vite (bundler moderno para frontend)
6. Instale a extensão 'C/C++' e 'Python' no VS Code

2.3 IDE Recomendada

Visual Studio Code (VS Code) como IDE principal. Configurar as extensões:

- C/C++ Extension Pack (Microsoft) - IntelliSense, debug, formatação
- Python (Microsoft) - ambiente virtual, pylint, pytest integrado
- x86 and x86_64 Assembly (13xforever) - syntax highlight para .asm
- REST Client - testa a API diretamente do editor
- Live Server - reload automático do frontend durante desenvolvimento
- GitLens - controle de versão avançado

2.4 Compiladores e Comandos Essenciais

```
# Compilar módulo C
gcc -Wall -Wextra -g -o simulador simulador.c

# Compilar como biblioteca compartilhada (carregada pelo Python
gcc -shared -fPIC -o libsimulador.so cpu.c memory.c

# Compilar Assembly com NASM
nasm -f elf64 operacoes.asm -o operacoes.o
gcc -no-pie operacoes.o main.c -o programa

# Debugar com GDB
gcc -g programa.c -o programa
gdb ./programa

# Checar vazamentos de memória com Valgrind
valgrind --leak-check=full ./programa
```

3. ESTRUTURA COMPLETA DO PROJETO

```
/simulador/
├── frontend/                    # Interface visual Web
│   ├── index.html               # Página principal com abas
│   ├── style.css                # Estilos da interface
│   ├── main.js                  # Lógica JavaScript principal
│   ├── components/              # Componentes reutilizáveis (se usar React)
│   │   ├── CPUSimulator.jsx
│   │   ├── MemoryMonitor.jsx
│   │   └── Charts.jsx
│   └── assets/                  # Ícones, imagens
│
```

```
├── backend/                    # Servidor Python (API + WebSocket)
│   ├── main.py                 # Entrada FastAPI
│   ├── api/
│   │   ├── cpu_routes.py       # Endpoints do CPU Simulator
│   │   └── memory_routes.py    # Endpoints do Memory Monitor
│   ├── core/
│   │   ├── cpu_bridge.py       # ctypes → libsimulador.so
│   │   └── memory_bridge.py
│   └── ws/
│       └── realtime.py         # WebSocket handler
│
├── c_modules/                  # Código C do simulador
│   ├── cpu.h / cpu.c           # CPU: registradores, ULA, UC, pipeline
│   ├── memory.h / memory.c     # Memória: segmentos, malloc, free
│   ├── flags.h / flags.c       # FLAGS: ZF, SF, CF, OF
│   ├── attack_sim.h / .c       # Simulação de ataques (buffer overflow)
│   └── Makefile                # Compila tudo em libsimulador.so
│
├── assembly/                   # Arquivos Assembly (.asm)
│   ├── operacoes.asm           # ADD, SUB, MUL chamados pelo C
│   ├── stack_frame.asm         # Demonstra stack frame real
│   └── README.md               # Explicação de cada rotina
│
├── docs/                       # Documentação acadêmica
│   ├── introducao.md
│   ├── metodologia.md
│   ├── desenvolvimento.md
│   ├── resultados.md
│   └── conclusao.md
│
├── tests/                      # Testes automatizados
│   ├── test_cpu.py             # pytest para endpoints CPU
```

4. DESENVOLVIMENTO EM FASES

FASE 1 - PLANEJAMENTO E REQUISITOS

O que fazer na Fase 1

- Escrever um documento de requisitos: listando as funcionalidades obrigatórias, opcionais e restrições.
- Criar o fluxo: como o usuário interage com o sistema.
- Definir os casos de uso principais: 'Usuário clica em Passo ->CPU executa uma instrução ->interface atualiza registradores';
- Criar o repositório Git: git init simulador e cd simulador.
- Criar o README.md do projeto com todas as instruções básicas

FASE 2 - Ambiente de Desenvolvimento

Checklist de Configuração

- Testar gcc: gcc --version deve retornar 11.x ou superior.
- Testar nasm: nasm --version deve retornar 2.15 ou superior.
- Testar python3
- Criar e ativar ambiente virtual Python: python3 -m venv venv && source venv/bin/activate.
- Instalar as dependências: pip install fastapi uvicorn websockets pytest.
- Testar o servidor: uvicorn backend.main:app --reload – deve iniciar sem erros.
- Testar o compilador C com um 'Hello World' dentro de c_modules/.
- Testar o NASM com uma rotina simples que retorna um valor inteiro.

```
# Arquivo: c_modules/hello_test.c
#include <stdio.h>
int main() {
    printf("compilando\n");
    return 0;
}
# Compilar e rodar
gcc c_modules/hello_test.c -o hello_test && ./hello_test
```

Ponto de Verificação – NÃO avançar sem:

- gcc, nasm, python3 instalados e funcionando
- Servidor FastAPI iniciando sem erros
- Frontend abrindo no navegador (mesmo que vazio)
- Git com pelo menos 2 commits

FASE 3 - CPU Simulador - Lógica em C
Implementar o simulador: registradores, ULA, UC e o ciclo Fetch-Decode-Execute.

4.3 Módulo CPU em C

Começar pelo arquivo de cabeçalho (cpu.h) definindo as estruturas de dados.

4.3.1 Estruturas de Dados – cpu.h

```
// c_modules/cpu.h
#ifndef CPU_H
```

```
#define CPU_H
#include <stdint.h>
#include <stdbool.h>
// REGISTRADORES / 64 BITS
typedef struct {
    uint64_t rax;   // Acumulador / retorno de função
    uint64_t rbx;   // Base
    uint64_t rcx;   // Contador
    uint64_t rdx;   // Dados
    uint64_t rsi;   // Source Index
    uint64_t rdi;   // Destination Index
    uint64_t rsp;   // Stack Pointer
    uint64_t rbp;   // Base Pointer (frame)
    uint64_t rip;   // Instruction Pointer (PC)
    uint64_t r8;    // Registradores extras x86-64
    uint64_t r9;
    uint64_t r10;
    uint64_t r11;
} Registers;
// FLAGS (EFLAGS)
typedef struct {
    bool ZF;  // Zero Flag: resultado == 0
    bool SF;  // Sign Flag: resultado negativo
    bool CF;  // Carry Flag: overflow sem sinal
    bool OF;  // Overflow Flag: overflow com sinal
    bool PF;  // Parity Flag: paridade
} Flags;
// INSTRUÇÃO
typedef enum {
    INSTR_MOV,  INSTR_ADD,  INSTR_SUB,
```

```
    INSTR_MUL,  INSTR_DIV,  INSTR_CMP,
    INSTR_JMP,  INSTR_JE,   INSTR_JNE,
    INSTR_PUSH, INSTR_POP,  INSTR_CALL,
    INSTR_RET,  INSTR_NOP,  INSTR_HLT,
} InstrType;
typedef struct {
    InstrType type;
    uint64_t  operand1;
    uint64_t  operand2;
    char      mnemonic[32];  // ex: "ADD RAX, RBX"
} Instruction;
// ESTADO DA CPU
typedef struct {
    Registers regs;
    Flags     flags;
    Instruction pipeline[4];  // IF, ID, EX, WB
    uint64_t  cycle_count;
    uint64_t  instr_count;
    bool      halted;
} CPU;
//PROTÓTIPOS
void cpu_init(CPU *cpu);
void cpu_reset(CPU *cpu);
void cpu_fetch(CPU *cpu, Instruction *memory, int mem_size);
void cpu_decode(CPU *cpu);
void cpu_execute(CPU *cpu);
void cpu_step(CPU *cpu, Instruction *memory, int mem_size);
void cpu_run(CPU *cpu, Instruction *memory, int mem_size);
void ula_operate(CPU *cpu, InstrType op, uint64_t a, uint64_t b);
void update_flags(CPU *cpu, int64_t result);
```

```
#endif // CPU_H
```

4.3.2 Implementação da ULA e Ciclo – cpu.c

```
// c_modules/cpu.c
#include "cpu.h"
#include <string.h>
#include <stdio.h>
// Inicializa todos os registradores e flags a zero
void cpu_init(CPU *cpu) {
    memset(cpu, 0, sizeof(CPU));
    cpu->regs.rsp = 0x7FFF0000;
}
void cpu_reset(CPU *cpu) { cpu_init(cpu); }
// FETCH: carrega a próxima instrução apontada pelo RIP
void cpu_fetch(CPU *cpu, Instruction *memory, int mem_size) {
    if (cpu->halted) return;
    int idx = (int)(cpu->regs.rip);
    if (idx >= mem_size) { cpu->halted = true; return; }
    cpu->pipeline[0] = memory[idx]; // estágio IF
    cpu->regs.rip++;                // avança o PC
}
// DECODE: move a instrução do IF para o ID
void cpu_decode(CPU *cpu) {
    cpu->pipeline[1] = cpu->pipeline[0]; // IF → ID
}
// Atualiza FLAGS com base no resultado
void update_flags(CPU *cpu, int64_t result) {
    cpu->flags.ZF = (result == 0);
    cpu->flags.SF = (result < 0);
    cpu->flags.CF = false;  // simplificado
    cpu->flags.OF = false;  // simplificado
```

```
}
// EXECUTE: executa a instrução que está no estágio EX
void cpu_execute(CPU *cpu) {
    Instruction *instr = &cpu->pipeline[1];
    int64_t result = 0;
    switch (instr->type) {
        case INSTR_MOV:
            cpu->regs.rax = instr->operand1;
            break;
        case INSTR_ADD:
            result = (int64_t)(cpu->regs.rax + instr->operand1);
            cpu->regs.rax = (uint64_t)result;
            update_flags(cpu, result);
            break;
        case INSTR_SUB:
            result = (int64_t)(cpu->regs.rax - instr->operand1);
            cpu->regs.rax = (uint64_t)result;
            update_flags(cpu, result);
            break;
        case INSTR_MUL:
            result = (int64_t)(cpu->regs.rax * instr->operand1);
            cpu->regs.rax = (uint64_t)result;
            update_flags(cpu, result);
            break;
        case INSTR_HLT:
            cpu->halted = true;
            break;
        case INSTR_NOP:
        default:
            break;  // Não faz nada - parar
```

```
    }
    cpu->instr_count++;
    cpu->cycle_count += 3; // IF + ID + EX = 3 ciclos
}
// STEP: executa UM ciclo completo (Fetch → Decode → Execute)
void cpu_step(CPU *cpu, Instruction *memory, int mem_size) {
    cpu_fetch(cpu, memory, mem_size);
    cpu_decode(cpu);
    cpu_execute(cpu);
}
// RUN: executa até HALT ou fim da memória
void cpu_run(CPU *cpu, Instruction *memory, int mem_size) {
    while (!cpu->halted) {
        cpu_step(cpu, memory, mem_size);
    }
}
```

4.3.3 Testando o CPU em C puro (antes de integrar com Python)

```
// c_modules/test_cpu_main.c
#include "cpu.h"
#include <stdio.h>
int main() {
    CPU cpu;
    cpu_init(&cpu);
    // Programa simples: MOV RAX, 10 → ADD RAX, 5 → HLT
    Instruction program[] = {
        { INSTR_MOV, 10, 0, "MOV RAX, 10" },
        { INSTR_ADD,  5, 0, "ADD RAX, 5"  },
        { INSTR_HLT,  0, 0, "HLT"         },
    };
    cpu_run(&cpu, program, 3);
```

```
    printf("RAX = %llu\n", cpu.regs.rax);     // Esperado: 15
    printf("Ciclos = %llu\n", cpu.cycle_count);
    printf("ZF = %d\n", cpu.flags.ZF);
    return 0;
}
// gcc cpu.c test_cpu_main.c -o test_cpu && ./test_cpu
```

FASE 4 - Memory Monitor - Hierarquia de memória

4.4 Módulo Memory em C

A memória de um processo Linux é dividida em segmentos bem definidos.

4.4.1 Mapa de Memória de um Processo

Segmento	O que armazena e como funciona
Text Segment	Código executável do programa (instruções). Somente leitura. Endereços baixos (~0x400000 no Linux x86-64).
Data Segment	Variáveis globais e estáticas inicializadas (int x = 5; em escopo global).
BSS Segment	Variáveis globais e estáticas NÃO inicializadas (int y; em escopo global). Preenchido com zeros pelo OS.
Heap	Memória alocada dinamicamente em runtime (malloc, calloc, realloc). Cresce para cima (endereços crescentes).
Stack	Variáveis locais, endereços de retorno e parâmetros de funções. Cresce para baixo (endereços decrescentes). Gerenciado automaticamente.
Kernel Space	Parte superior do espaço de endereço reservada ao kernel. Inacessível diretamente pelo processo. Simulada como área proibida.

4.4.2 Estrutura memory.h

```
// c_modules/memory.h
#ifndef MEMORY_H
#define MEMORY_H
#include <stdint.h>
```

```
#include <stddef.h>
#define MEM_TOTAL     (64 * 1024)  // 64 KB simulados
#define TEXT_BASE     0x0000
#define DATA_BASE     0x2000
#define BSS_BASE      0x3000
#define HEAP_BASE     0x4000
#define HEAP_MAX      0xC000
#define STACK_BASE    0xFFFF
#define STACK_MIN     0xC000
typedef struct HeapBlock {
    uint32_t          address;
    size_t            size;
    int               is_free;
    struct HeapBlock  *next;
} HeapBlock;
typedef struct {
    uint8_t    raw[MEM_TOTAL];   // Memória bruta simulada
    uint32_t   heap_top;         // Topo atual do heap
    uint32_t   stack_top;        // Topo atual da stack
    HeapBlock *heap_list;        // Lista encadeada de blocos
    size_t     heap_used;        // Bytes em uso no heap
    size_t     stack_used;       // Bytes em uso na stack
} Memory;
void     mem_init(Memory *mem);
uint32_t mem_malloc(Memory *mem, size_t size);   // retorna endereço
void     mem_free(Memory *mem, uint32_t addr);
uint32_t mem_realloc(Memory *mem, uint32_t addr, size_t new_size);
void     mem_stack_push(Memory *mem, uint64_t value);
uint64_t mem_stack_pop(Memory *mem);
void     mem_dump(Memory *mem);  // imprime estado no terminal
```

```
#endif
```

4.4.3 Implementação de malloc - memory.c

```
// c_modules/memory.c
#include "memory.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
void mem_init(Memory *mem) {
    memset(mem->raw, 0, MEM_TOTAL);
    mem->heap_top   = HEAP_BASE;
    mem->stack_top  = STACK_BASE;
    mem->heap_list  = NULL;
    mem->heap_used  = 0;
    mem->stack_used = 0;
}
// Aloca 'size' bytes no heap simulado
// Retorna o endereço simulado ou 0 se falhar
uint32_t mem_malloc(Memory *mem, size_t size) {
    // Primeiro: tenta reutilizar bloco livre (best-fit simplificado)
    HeapBlock *blk = mem->heap_list;
    while (blk) {
        if (blk->is_free && blk->size >= size) {
            blk->is_free = 0;
            mem->heap_used += size;
            return blk->address;
        }
        blk = blk->next;
    }
    // Sem bloco livre: aloca novo espaço
    if (mem->heap_top + size > HEAP_MAX) return 0; // Sem espaço
```

```
    HeapBlock *new_blk = (HeapBlock *)malloc(sizeof(HeapBlock));
    new_blk->address = mem->heap_top;
    new_blk->size    = size;
    new_blk->is_free = 0;
    new_blk->next    = mem->heap_list;
    mem->heap_list   = new_blk;
    mem->heap_top  += (uint32_t)size;
    mem->heap_used += size;
    return new_blk->address;
}
// Libera o bloco no endereço dado
void mem_free(Memory *mem, uint32_t addr) {
    HeapBlock *blk = mem->heap_list;
    while (blk) {
        if (blk->address == addr && !blk->is_free) {
            blk->is_free = 1;
            mem->heap_used -= blk->size;
            memset(&mem->raw[addr], 0, blk->size); // limpa bytes
            return;
        }
        blk = blk->next;
    }
}
// Push na stack simulada
void mem_stack_push(Memory *mem, uint64_t value) {
    mem->stack_top -= sizeof(uint64_t);
    memcpy(&mem->raw[mem->stack_top], &value, sizeof(uint64_t));
    mem->stack_used += sizeof(uint64_t);
}
uint64_t mem_stack_pop(Memory *mem) {
```

```
    uint64_t value;
    memcpy(&value, &mem->raw[mem->stack_top], sizeof(uint64_t));
    mem->stack_top  += sizeof(uint64_t);
    mem->stack_used -= sizeof(uint64_t);
    return value;
}
```

FASE 5 - Integração C + Assembly x86-64

4.5 Como C Chama Assembly

A convenção de chamada do Linux x86-64 (System V AMD64 ABI) define como argumentos são passados e como o retorno acontece:

- Primeiro argumento: RDI. Segundo: RSI. Terceiro: RDX. Quarto: RCX. Quinto: R8. Sexto: R9.
- Valor de retorno: RAX (inteiro) ou XMM0 (float).
- Registradores que a função chamada DEVE preservar: RBX, RBP, R12-R15.
- Registradores que podem ser sobrescritos livremente: RAX, RCX, RDX, RSI, RDI, R8-R11.

4.5.1 Arquivo Assembly – operacoes.asm

```
; assembly/operacoes.asm
; Sintaxe NASM (Intel), x86-64
section .text
    global asm_add       ; exporta para o linker
    global asm_multiply
    global asm_fib
; long asm_add(long a, long b)
; a → RDI,  b → RSI,  retorno → RAX
asm_add:
    mov rax, rdi         ; RAX = a
    add rax, rsi         ; RAX = a + b
    ret                  ; retorna RAX
```

```
; long asm_multiply(long a, long b)
asm_multiply:
    mov rax, rdi        ; RAX = a
    imul rax, rsi       ; RAX = a * b  (signed)
    ret
; long asm_fib(int n)  — Fibonacci iterativo
; n → RDI
asm_fib:
    cmp rdi, 1
    jle .base           ; se n <= 1, retorna n
    mov rax, 0          ; fib(0) = 0
    mov rcx, 1          ; fib(1) = 1
    mov rdx, rdi        ; contador = n
.loop:
    mov rsi, rcx        ; rsi = fib(n-1)
    add rcx, rax        ; rcx = fib(n-1) + fib(n-2)
    mov rax, rsi
    dec rdx
    cmp rdx, 1
    jg .loop
    mov rax, rcx
    ret
.base:
    mov rax, rdi
    ret
```

4.5.2 Chamando Assembly a partir do C

```
// c_modules/asm_bridge.c
#include <stdio.h>
#include <stdint.h>
// Declaração: diz ao C que essas funções existem em Assembly
```

```
extern long asm_add(long a, long b);
extern long asm_multiply(long a, long b);
extern long asm_fib(int n);
int main() {
    long soma = asm_add(15, 27);
    printf("15 + 27 = %ld\n", soma);         // 42
    long produto = asm_multiply(6, 7);
    printf("6 x 7 = %ld\n", produto);        // 42
    long fib10 = asm_fib(10);
    printf("Fibonacci(10) = %ld\n", fib10);  // 55
    return 0;
}
// Compilar:
// nasm -f elf64 assembly/operacoes.asm -o operacoes.o
// gcc -no-pie operacoes.o c_modules/asm_bridge.c -o demo_asm
// ./demo_asm
```

4.5.3 Stack Frame em Assembly – Para visualização educacional

```
; assembly/stack_frame.asm
; Demonstra o que acontece na stack ao chamar uma função
section .text
    global demo_stack_frame
; void demo_stack_frame(int a, int b)
demo_stack_frame:
    push rbp              ; salva o Base Pointer anterior
    mov  rbp, rsp         ; rbp aponta para o topo da stack (frame atual)
    sub  rsp, 16          ; reserva 16 bytes para variáveis locais
    ; [rbp - 4]  = variável local 'x' = a + b
    mov eax, edi          ; eax = a
    add eax, esi          ; eax = a + b
    mov [rbp - 4], eax    ; armazena na stack
```

```
    ; Cleanup e retorno
    mov eax, [rbp - 4]   ; retorno = x
    leave                ; equivale a: mov rsp, rbp / pop rbp
    ret
; Estrutura do Stack Frame gerado:
;  endereço alto
;  [rbp + 8]  → endereço de retorno (push automático pelo CALL)
;  [rbp + 0]  → rbp anterior (salvo pelo push rbp)
;  [rbp - 4]  → variável local x
;  [rbp - 8]  → padding (alinhamento 16 bytes)
;  endereço baixo ← rsp aponta aqui
```

FASE 6 - Interface Visual Web

4.6 Backend API com FastAPI (Python)

```
# backend/main.py
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
import ctypes, json, asyncio
app = FastAPI(title="CPU Simulator API")
# Permite o frontend (outra porta) acessar o backend
app.add_middleware(CORSMiddleware, allow_origins=["*"],
    allow_methods=["*"], allow_headers=["*"])
# Carrega a biblioteca C compilada
lib = ctypes.CDLL("./c_modules/libsimulador.so")
# Define tipos de retorno das funções C
lib.cpu_get_rax.restype    = ctypes.c_uint64
lib.cpu_get_cycles.restype = ctypes.c_uint64
lib.mem_get_heap_used.restype = ctypes.c_size_t
@app.get("/api/cpu/step")
```

```
async def cpu_step():
    lib.cpu_step_global()   # avança 1 ciclo na CPU simulada
    return {
        "rax":    lib.cpu_get_rax(),
        "rbx":    lib.cpu_get_rbx(),
        "rip":    lib.cpu_get_rip(),
        "cycles": lib.cpu_get_cycles(),
        "zf":     bool(lib.cpu_get_zf()),
        "halted": bool(lib.cpu_is_halted()),
    }
@app.get("/api/memory/status")
async def memory_status():
    return {
        "heap_used":  lib.mem_get_heap_used(),
        "stack_used": lib.mem_get_stack_used(),
        "heap_pct":   lib.mem_get_heap_pct(),
    }
@app.websocket("/ws/realtime")
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    while True:
        data = {
            "rax":       lib.cpu_get_rax(),
            "heap_used": lib.mem_get_heap_used(),
            "cycles":    lib.cpu_get_cycles(),
        }
        await ws.send_text(json.dumps(data))
        await asyncio.sleep(0.5)  # atualiza 2x por segundo
# Rodar: uvicorn backend.main:app --reload --port 8000
```

4.6.2 Frontend HTML – Estrutura das Abas

```
<!-- frontend/index.html -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Simulador de Arquitetura de Computadores</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Simulador de Arquitetura de Computadores</h1>
    <nav class="tabs">
      <button class="tab active" data-tab="cpu">CPU Simulator</button>
      <button class="tab" data-tab="memory">Memory Monitor</button>
      <button class="tab" data-tab="graphs">Gráficos</button>
      <button class="tab" data-tab="attacks">Ataques</button>
    </nav>
  </header>
  <!-- ABA CPU -->
  <section id="tab-cpu" class="tab-content active">
    <div class="registers-grid">
      <div class="reg-card"><span class="reg-name">RAX</span><span
id="reg-rax" class="reg-val">0x0000000000000000</span></div>
      <div class="reg-card"><span class="reg-name">RBX</span><span
id="reg-rbx" class="reg-val">0x0000000000000000</span></div>
      <div class="reg-card"><span class="reg-name">RIP</span><span id="reg-rip"
class="reg-val">0x0000000000000000</span></div>
      <div class="reg-card"><span class="reg-name">RSP</span><span
id="reg-rsp" class="reg-val">0x0000000000000000</span></div>
    </div>
```

```
    <div class="flags-row">
      <span class="flag" id="flag-zf">ZF=0</span>
      <span class="flag" id="flag-sf">SF=0</span>
      <span class="flag" id="flag-cf">CF=0</span>
      <span class="flag" id="flag-of">OF=0</span>
    </div>
    <div class="pipeline-visual">
      <div class="stage" id="stage-if">IF<br><small id="if-instr">—</small></div>
      <div class="stage" id="stage-id">ID<br><small id="id-instr">—</small></div>
      <div class="stage" id="stage-ex">EX<br><small id="ex-instr">—</small></div>
      <div class="stage" id="stage-wb">WB<br><small
id="wb-instr">—</small></div>
    </div>
    <button id="btn-step" onclick="cpuStep()">Passo</button>
    <button id="btn-run"  onclick="cpuRun()">Executar</button>
    <button id="btn-reset" onclick="cpuReset()">Reset</button>
  </section>
  <!-- ABA MEMÓRIA -->
  <section id="tab-memory" class="tab-content">
    <div class="memory-map" id="memory-map"></div>
    <div class="mem-controls">
      <input type="number" id="malloc-size" placeholder="Tamanho (bytes)">
      <button onclick="doMalloc()">malloc()</button>
      <button onclick="doFree()">free()</button>
    </div>
  </section>
  <script src="main.js"></script>
</body>
</html>
```

FASE 7 - Gráficos em tempo real

4.7 Gráficos com Chart.js via WebSocket

```
// frontend/main.js - Seção de Gráficos
// 1. Importe o Chart.js no HTML via CDN:
// <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
// 2. Configure os gráficos
const MAX_POINTS = 50; // últimos 50 pontos
const labels  = [];
const heapData = [];
const cpuData  = [];
const ctxHeap = document.getElementById('chart-heap').getContext('2d');
const heapChart = new Chart(ctxHeap, {
  type: 'line',
  data: {
    labels,
    datasets: [{
      label: 'Heap Usado (bytes)',
      data: heapData,
      borderColor: '#2E86C1',
      backgroundColor: 'rgba(46,134,193,0.1)',
      tension: 0.3, fill: true,
    }]
  },
  options: { animation: false, scales: { y: { beginAtZero: true } } }
});
// 3. Conecta ao WebSocket e atualiza gráficos
const ws = new WebSocket('ws://localhost:8000/ws/realtime');
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  const now  = new Date().toLocaleTimeString();
  // Mantém apenas os últimos MAX_POINTS
```

```
  if (labels.length >= MAX_POINTS) {
    labels.shift(); heapData.shift(); cpuData.shift();
  }
  labels.push(now);
  heapData.push(data.heap_used);
  cpuData.push(data.cycles);
  heapChart.update();
  // Atualiza registradores na interface
  document.getElementById('reg-rax').textContent =
    '0x' + data.rax.toString(16).padStart(16, '0').toUpperCase();
};
ws.onerror = () => console.error('WebSocket: erro de conexão');
ws.onclose = () => console.log('WebSocket: conexão encerrada');
```

Quatro Gráficos para Implementar

1. Heap Usage over Time - line chart -> conecta a mem_get_heap_used() via WebSocket
2. Stack Usage - bar chart -> conecta a mem_get_stack_used()
3. Instructions Per Second - line chart -> calcula delta de instr_count / intervalo
4. CPU Cycle Distribution - doughnut chart -> mostra proporção de tipos de instrução

FASE 8 - Simulação de ataques computacionais

4.8 Buffer Overflow - Demonstração Didática

O ataque de Buffer Overflow acontece quando um programa escreve além dos limites de um buffer alocado na stack, sobrescrevendo o endereço de retorno e redirecionando o fluxo de execução.

```
// c_modules/attack_sim.c
// gcc -fno-stack-protector -z execstack -no-pie attack_sim.c -o demo_attack
#include <stdio.h>
#include <string.h>
```

```
// Função que seria executada pelo atacante
void funcao_maliciosa() {
    printf("[ATAQUE] Controle de fluxo sequestrado!\n");
}
// Função vulnerável: não verifica o tamanho da entrada
void funcao_vulneravel(char *entrada) {
    char buffer[64];  // apenas 64 bytes
    // strcpy NÃO verifica tamanho — NUNCA use em código real
    strcpy(buffer, entrada);  // overflow se entrada > 64 bytes
    printf("Buffer: %s\n", buffer);
}
int main() {
    // Entrada segura
    funcao_vulneravel("Olá Mundo");
    // Entradas que causam overflow (didático):
    // funcao_vulneravel("A" * 80);  // sobrescreve stack
    return 0;
}
// No simulador: detectar o overflow comparando
// o endereço de retorno atual com o salvo antes da chamada.
```

4.8.1 Detectando Overflow no Simulador

```
// c_modules/attack_sim.h — funções de detecção
// Salva o endereço de retorno antes de uma chamada
uint64_t save_return_address(Memory *mem);
// Verifica se o endereço de retorno foi modificado
int check_stack_integrity(Memory *mem, uint64_t saved_addr);
// No frontend: exiba um alerta vermelho quando check retornar 1
// e mostre qual byte foi sobrescrito no Memory Monitor.
```

FASE 9 - Testes e Validação

4.9 Estratégia de Testes

Tipo de Teste	Como executar
Testes de Unidade C	Usar CUnit ou assert() para testar ULA, flags e funções de memória. Execute: gcc test_ula.c cpu.c -o test && ./test
Testes de API Python	Usar pytest para testar todos os endpoints REST. Execute: pytest tests/ -v
Testes de Integração	Executar um programa completo na CPU simulada e verificar se o resultado em RAX está correto.
Testes de WebSocket	Usar a biblioteca websocket-client do Python para verificar que o WS envia dados válidos.
Testes de Overflow	Alocar 64 bytes e escrever 80: verifique que o simulador detecta e reporta o overflow.
Testes de UI	Testar manualmente cada botão e verificar que os valores exibidos batem com os calculados.

4.9.1 Exemplo de Teste pytest para a API

```
# tests/test_cpu.py
import requests, pytest
BASE = "http://localhost:8000"
def test_cpu_step_retorna_registradores():
    r = requests.get(f"{BASE}/api/cpu/step")
    assert r.status_code == 200
    data = r.json()
    assert "rax"    in data
    assert "rip"    in data
    assert "cycles" in data
    assert isinstance(data["halted"], bool)
```

```
def test_memory_status():
    r = requests.get(f"{BASE}/api/memory/status")
    assert r.status_code == 200
    data = r.json()
    assert "heap_used" in data
    assert data["heap_used"] >= 0
# Rodar: pytest tests/test_cpu.py -v
```

FASE 10 - Documentação Acadêmica e Entrega Final

4.10 Estrutura do Documento Acadêmico

Seção	O que deve conter
1. Introdução	Apresentar o problema, a motivação, os objetivos gerais e específicos, e a estrutura do trabalho.
2. Referencial Teórico	Revisar: ciclo Fetch-Decode-Execute, hierarquia de memória, convenção de chamada x86-64, pipeline, buffer overflow. Cite fontes (Tanenbaum, Patterson & Hennessy).
3. Metodologia	Descrever as tecnologias escolhidas e por quê, o processo de desenvolvimento iterativo, as fases do projeto e como cada decisão técnica foi tomada.
4. Desenvolvimento	Documento de cada módulo: CPU, Memória, Assembly, Backend, Frontend. Inclua diagramas, trechos de código comentados e print screens da interface.
5. Resultados	Apresentação: prints da interface funcionando, gráficos gerados, saída dos testes (pytest -v), comparação entre comportamento simulado e comportamento real.

Seção	O que deve conter
6. Conclusão	O objetivo foi atingido? O simulador ajuda no aprendizado? Quais limitações existem? Quais são os trabalhos futuros?
Referências	google acadêmico, livros e Science Direct

REFERÊNCIAS

[1] TANENBAUM, Andrew S. Organização Estruturada de Computadores. 6ª ed. Pearson, 2013.

[2] PATTERSON, D.; HENNESSY, J. Organização e Projeto de Computadores. 5ª ed. Elsevier, 2017.

[3] INTEL. Intel 64 and IA-32 Architectures Software Developer's Manual. Intel Corp., 2023.

[4] GCC Team. GCC, the GNU Compiler Collection Documentation. gnu.org, 2023.

[5] ALEPH ONE. Smashing the Stack for Fun and Profit. Phrack Magazine, n.49, 1996.